

Decentralized Password Management: Building a Web3-Native Vault with AES-256-GCM, IPFS, and BNB Chain

How to combine client-side AES-256-GCM encryption, Web Crypto API deterministic key derivation, IPFS storage, BNB Chain smart contracts, and GraphQL subgraph indexing for 100% user-owned password security.

In an era dominated by digital accounts, the security of our credentials has never been more critical. Yet, standard solutions continue to rely on centralized frameworks. Traditional password managers—while convenient—store credentials on centralized cloud databases or rely on master keys that present a single point of failure. If the database is breached, or the service provider's servers go down, users face catastrophic security risks or total loss of access.

Web3 introduces a new paradigm: self-custody and cryptographic identity. What if we could build a password manager where passwords are encrypted client-side using industry-standard **AES-256-GCM**, stored on a decentralized filesystem (IPFS), registered on a secure blockchain network (BNB Chain), and decrypted *only* by deriving keys client-side directly from the user's Web3 wallet?

This is the foundation of **SecureVault**. By upgrading from external key-node networks to pure client-side Web Crypto API encryption, SecureVault guarantees zero third-party trust, zero master password friction, and total user sovereignty over digital credentials.

The Technical Architecture Stack

To eliminate centralized vulnerability vectors and eliminate dependency on external key management services, SecureVault leverages a production-grade Web3 stack:

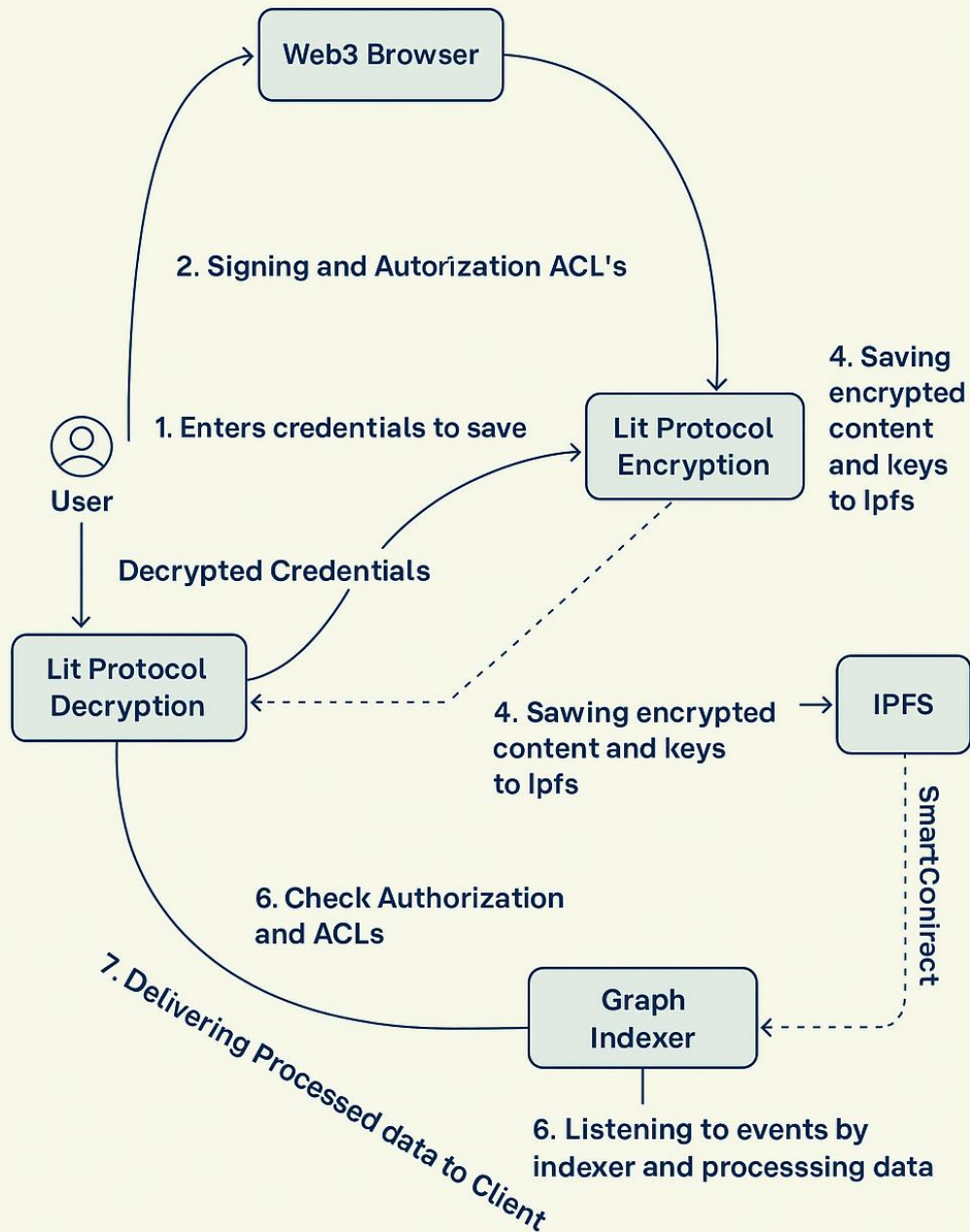
Component	Technology Used	Primary Role & Purpose
Frontend UI	Next.js 14 & Ant Design (Antd)	Provides a responsive dashboard for password management, generation, and clipboard integration.
Web3 Client	Ethers.js, Wagmi & RainbowKit	Manages wallet connection, network switching to BSC Testnet, and transaction signing.
Key Derivation	MetaMask Signature + SHA-256	Derives a 256-bit AES CryptoKey deterministically from the user's

		wallet signature.
Symmetric Encryption	Web Crypto API (AES-256-GCM)	Performs hardware-accelerated client-side encryption with 96-bit random IVs in the browser.
Decentralized Storage	IPFS (via Pinata)	Stores encrypted JSON payloads off-chain to keep gas costs minimal.
On-Chain Registry	Solidity (Hardhat on BNB Chain)	KeyManager smart contract logs IPFS CIDs tied immutably to user wallet addresses.

Workflow Architecture: The End-to-End Data Pipeline

Plaintext passwords never touch a network socket. The workflow below illustrates how key derivation, AES encryption, IPFS pinning, and blockchain registration coordinate seamlessly:

Lit SecureVault Workflow



© Samir Jumade

Figure 1: End-to-end cryptographic pipeline and smart contract synchronization in SecureVault.

- **1. Wallet Authentication:** The user connects their Web3 wallet (MetaMask) to the BSC Testnet network.
- **2. Deterministic Key Derivation:** The user signs a fixed, human-readable key-derivation message. The signature is digested via SHA-256 into a 32-byte AES-256-GCM CryptoKey using ``crypto.subtle.importKey``.
- **3. Client-Side AES-256-GCM Encryption:** The browser generates a 96-bit random Initialization Vector (IV) and encrypts the credential payload using the derived CryptoKey.
- **4. IPFS Archival:** The base64 ciphertext and IV are formatted into a JSON payload `{ ciphertext, iv, version: 'aes-gcm-v1' }` and pinned to IPFS via Pinata, returning a unique IPFS CID.
- **5. On-Chain Anchoring:** The client calls ``addKey(ipfsHash)`` or ``updateKey(id, ipfsHash)`` on the ``KeyManager.sol`` smart contract on BNB Chain Testnet.
- **6. Graph Subgraph Indexing:** The Graph indexer listens for ``KeyAdded`` / ``KeyUpdated`` events, fetches the JSON file via ``ipfs.cat``, and caches the payload fields for sub-second GraphQL retrieval.

Deep Dive 1: Deterministic Wallet-Based Key Derivation

A major bottleneck in Web3 encryption is secret key management. Traditional approaches require users to remember master passwords or rely on external key-share node networks. SecureVault solves this by deriving the encryption key directly from the user's Web3 wallet signature using the Web Crypto API (``client/lib/crypto.js``):

```
// client/lib/crypto.js - Deterministic Key Derivation
const KEY_DERIVATION_MESSAGE = (address) =>
[
  "SecureVault - Encryption Key Derivation",
  "",
  `Wallet: ${address.toLowerCase()}`,
  "",
  "Signing this message generates your AES-256-GCM encryption key.",
  "It is deterministic: signing again with the same wallet always",
  "produces the same key, so you can always decrypt your passwords.",
  "",
  "⚠️ Never share this signature with anyone.",
].join("\n");

export const deriveEncryptionKey = async (signer, address) => {
  const message = KEY_DERIVATION_MESSAGE(address);
  const signature = await signer.signMessage(message);

  const encoded = new TextEncoder().encode(signature);
  const digest = await crypto.subtle.digest("SHA-256", encoded);

  return crypto.subtle.importKey("raw", digest, { name: "AES-GCM" }, false, [
    "encrypt",
    "decrypt",
  ]);
}
```

```
  });  
};
```

Why is this approach cryptographically sound?

- **Deterministic & Stateless:** Signing the exact same message with the private key produces the exact same signature every time. This means no keys are ever saved to disk or transmitted over the wire.
- **Sole Control:** Only the holder of the wallet's private key can produce the required signature, guaranteeing that only the wallet owner can generate the matching AES key.
- **No External Node Dependencies:** Unlike threshold networks (e.g. Lit Protocol), this approach runs 100% locally inside the browser with zero network latency and no external server trust requirements.

Deep Dive 2: Native Web Crypto API AES-256-GCM Encryption

Galois/Counter Mode (GCM) provides both confidentiality and authenticated encryption (AEAD). Each encryption call generates a fresh 96-bit Initialization Vector (IV), ensuring identical plaintexts yield entirely distinct ciphertexts:

```
// client/lib/crypto.js - AES-256-GCM Encrypt & Decrypt  
export const encryptString = async (cryptoKey, plaintext) => {  
  const iv = crypto.getRandomValues(new Uint8Array(12)); // 96-bit random IV  
  const encoded = new TextEncoder().encode(plaintext);  
  
  const cipherBuffer = await crypto.subtle.encrypt(  
    { name: "AES-GCM", iv },  
    cryptoKey,  
    encoded  
  );  
  
  return {  
    ciphertext: toBase64(cipherBuffer),  
    iv: toBase64(iv),  
    version: "aes-gcm-v1",  
  };  
};  
  
export const decryptString = async (cryptoKey, { ciphertext, iv }) => {  
  const decrypted = await crypto.subtle.decrypt(  
    { name: "AES-GCM", iv: fromBase64(iv) },  
    cryptoKey,  
    fromBase64(ciphertext)  
  );  
  return new TextDecoder().decode(decrypted);  
};
```

```
};
```

The generated JSON payload (containing `ciphertext`, `iv`, and `version`) is pinned directly to IPFS, ensuring that anyone inspecting the IPFS hash sees only high-entropy, authenticated ciphertext.

Deep Dive 3: The Smart Contract & Subgraph Indexer

The **KeyManager.sol** contract deployed on BNB Chain Testnet acts as an immutable ledger mapping wallet addresses to array indices of IPFS CID hashes:

```
// SPDX-License-Identifier: MIT
pragma solidity 0.8.20;

contract KeyManager {
    struct Key {
        uint id;
        string ipfsHash;
        bool isDeleted;
    }

    event KeyAdded(uint id, string ipfsHash, address indexed owner);
    event KeyUpdated(uint id, string ipfsHash, address indexed owner);
    event KeyDeleted(uint id, address indexed owner);

    mapping(address => Key[]) keys;

    function addKey(string calldata _ipfsHash) public {
        keys[msg.sender].push(Key(keys[msg.sender].length, _ipfsHash, false));
        emit KeyAdded(keys[msg.sender].length - 1, _ipfsHash, msg.sender);
    }

    function updateKey(uint _id, string calldata _ipfsHash) public {
        keys[msg.sender][_id].ipfsHash = _ipfsHash;
        emit KeyUpdated(_id, _ipfsHash, msg.sender);
    }

    function softDeleteKey(uint _id) public {
        keys[msg.sender][_id].isDeleted = true;
        emit KeyDeleted(_id, msg.sender);
    }
}
```

To eliminate slow client-side IPFS gateway fetching, The Graph subgraph indexer listens for contract events, executes `ipfs.cat` during indexing, and caches the encrypted JSON payload for sub-second GraphQL retrieval.

Architectural Comparison: Centralized vs Lit Protocol vs AES-256-GCM

Feature	Centralized (1Password/LastPass)	Lit Protocol (Threshold Crypto)	SecureVault (Native AES-256-GCM)
Key Derivation	Master Password + PBKDF2	Threshold Secret Sharing (TSS) Node Network	MetaMask Signature + SHA-256 (Deterministic)
Encryption Location	Client/Cloud Server	Client + Lit Node Protocol	Pure Browser Web Crypto API (`crypto.subtle`)
Third-Party Trust	High (Cloud DB & Servers)	Medium (Lit Decentralized Nodes)	Zero (100% Client-Side Self-Custody)
Latency & Cost	Low / Subscription Fee	Medium / Node Verification Network	Sub-second / Low Gas on BNB Chain

User Interface & Visual Walkthrough

1. Dashboard & Web3 Wallet Connection

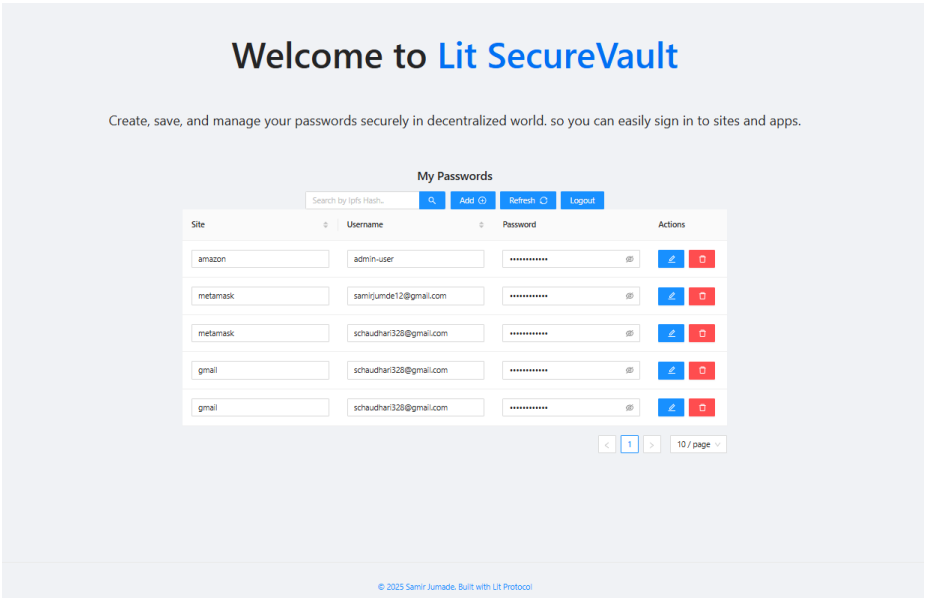


Figure 2: SecureVault Web3 Dashboard requesting wallet connection.

2. Password Encryption & Generation

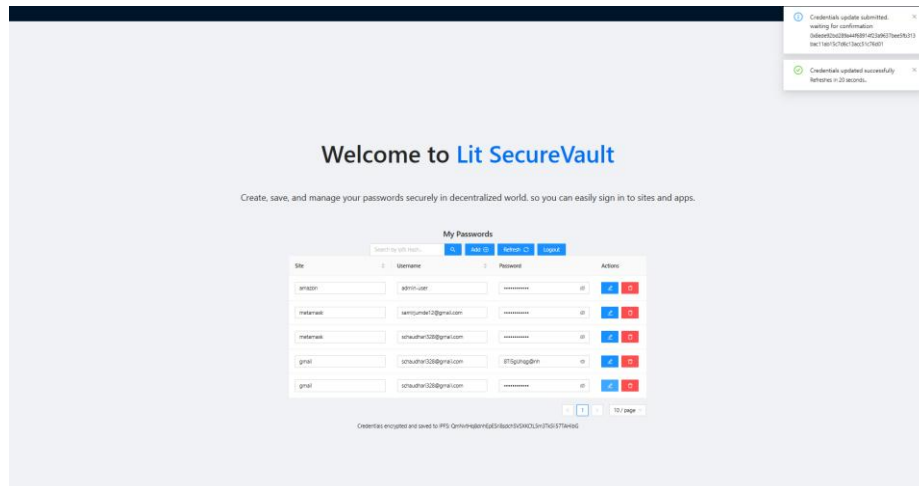


Figure 3: Save Password dialog with automatic password generator.

3. Masked Credentials Grid & Editing

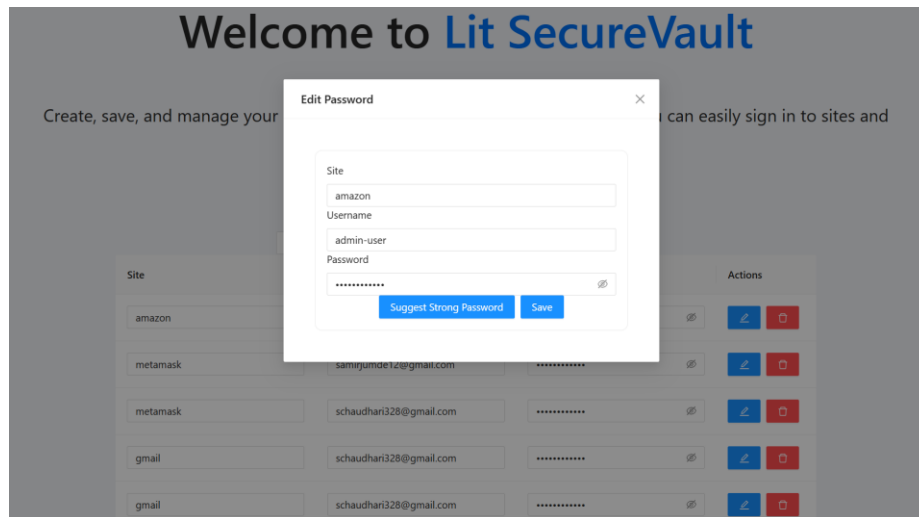


Figure 4: Masked credentials table with quick clipboard copy and editing.

4. On-Chain Smart Contract Updates

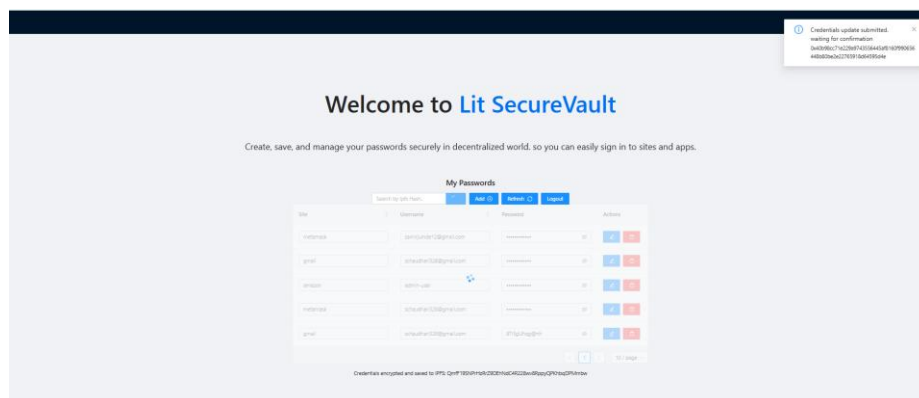


Figure 5: MetaMask transaction confirmation for on-chain registry updates.

Conclusion & The Future of Web3 Credentials

By combining browser-native AES-256-GCM encryption with deterministic wallet key derivation, IPFS storage, BNB Chain registry contracts, and GraphQL subgraph indexing, SecureVault proves that true cryptographic self-custody is not only possible—it is seamless and highly performant.

Users no longer need to rely on vulnerable centralized databases or external key-splitting servers. With SecureVault, your wallet is your master key, the blockchain is your registry, and your passwords belong solely to you.